

Feature Engineering, Web Scraping e APIs

Eduardo Adame

Ciência de Dados

15 de maio de 2026



O que é Feature Engineering?

Definição

- Processo de **criar, transformar e selecionar** variáveis a partir dos dados brutos
- Objetivo: fornecer ao modelo informação mais relevante e na forma certa
- Não é sobre o modelo — é sobre **o que você alimenta** nele

Por que importa?

- Modelos simples com boas features batem modelos complexos com features ruins
- Reduz overfitting ao eliminar ruído
- Diminui custo computacional

Tipos de transformação

- **Numéricas**: normalização, log, binning, interações
- **Categóricas**: encoding, target encoding, frequência
- **Temporais**: hora, dia da semana, sazonalidade
- **Texto**: TF-IDF, embeddings, contagem de tokens
- **Derivadas**: razões, diferenças, agregações por grupo



Listing 1: Normalização e transformações

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler, MinMaxScaler
4
5 # Padronização (média 0, desvio 1) - para modelos lineares e redes neurais
6 scaler = StandardScaler()
7 df["preco_std"] = scaler.fit_transform(df[["preco"]])
8 # Min-Max (intervalo [0, 1]) - para KNN, SVM
9 scaler_mm = MinMaxScaler()
10 df["preco_norm"] = scaler_mm.fit_transform(df[["preco"]])
11 # Log - para distribuições assimétricas (renda, preço de imóveis)
12 df["preco_log"] = np.log1p(df["preco"]) # log1p = log(1 + x), evita log(0)
13 # Binning - transformar contínua em ordinal
14 df["faixa_preco"] = pd.cut(df["preco"],
15                             bins=[0, 100, 500, np.inf],
16                             labels=["barato", "médio", "caro"])
```



Listing 2: One-Hot, Ordinal e Target Encoding

```
1 # One-Hot Encoding - para categorias sem ordem (cor, cidade)
2 df = pd.get_dummies(df, columns=["cidade"], drop_first=True)
3 # Ordinal Encoding - para categorias com ordem (faixa etária, grau)
4 from sklearn.preprocessing import OrdinalEncoder
5 oe = OrdinalEncoder(categories=[["baixo", "médio", "alto"]])
6 df["grau_enc"] = oe.fit_transform(df[["grau"]])
7 # Target Encoding - substitui categoria pela média do alvo
8 # (cuidado: vaza informação do alvo se não usar cross-fitting)
9 media_por_cidade = df.groupby("cidade")["preco"].mean()
10 df["cidade_target"] = df["cidade"].map(media_por_cidade)
11 # Frequência - para alta cardinalidade (CEP, ID de produto)
12 freq = df["produto_id"].value_counts(normalize=True)
13 df["produto_freq"] = df["produto_id"].map(freq)
```

Listing 3: Extrair informação de datas e criando interações

```
1 df["data"] = pd.to_datetime(df["data"])
2
3 # Decomposição temporal
4 df["hora"] = df["data"].dt.hour
5 df["dia_semana"] = df["data"].dt.dayofweek # 0 = segunda
6 df["mes"] = df["data"].dt.month
7 df["fim_de_semana"] = df["dia_semana"].isin([5, 6]).astype(int)
8
9 # Sazonalidade cíclica - hora não é linear (23h e 0h são próximas!)
10 df["hora_sin"] = np.sin(2 * np.pi * df["hora"] / 24)
11 df["hora_cos"] = np.cos(2 * np.pi * df["hora"] / 24)
12
13 # Features derivadas e interações
14 df["preco_por_m2"] = df["preco"] / df["area"]
15 df["idade_km"] = df["idade_carro"] * df["km_rodados"] # interação
16 df["media_por_grupo"] = df.groupby("cidade")["preco"] \
17     .transform("mean") # agregação sem vaziar teste
```

O problema de avaliar modelos



Por que não usar o treino para avaliar?

- O modelo **memoriza** os dados de treino
- Erro de treino $\ll$ erro real — *overfitting*
- Precisamos estimar o desempenho em dados **nunca vistos**

Holdout simples

- Separar treino / teste uma vez
- Rápido, mas **instável**: depende da divisão
- Com dados limitados, descarta informação valiosa

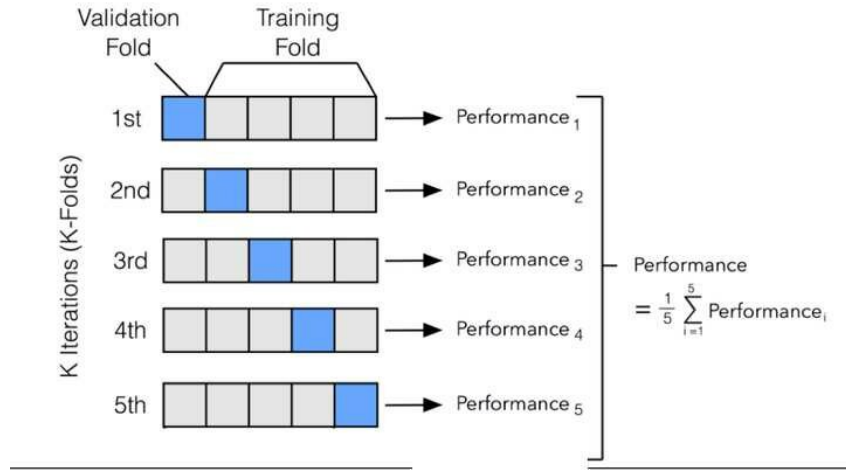
Cross-Validation

- Divide os dados em k partes (*folds*)
- Treina em $k - 1$ e testa no fold restante
- Repete k vezes, rotacionando o fold de teste
- Resultado: **média $\pm$ desvio** do erro nos k folds

Regra prática

$k = 5$ ou $k = 10$ são os valores mais usados. Estratifique para classificação desbalanceada.

k -Fold Cross-Validation



Listing 4: scikit-learn: avaliação e seleção de modelo

```
1 from sklearn.model_selection import cross_val_score, StratifiedKFold
2 from sklearn.ensemble import RandomForestClassifier
3 from sklearn.pipeline import Pipeline
4 from sklearn.preprocessing import StandardScaler
5 # Pipeline: pré-processamento + modelo em uma etapa
6 pipe = Pipeline([
7     ("scaler", StandardScaler()),
8     ("clf", RandomForestClassifier(n_estimators=100, random_state=42)),
9 ])
10 cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
11 scores = cross_val_score(pipe, X, y,
12                           cv=cv, scoring="f1_weighted")
13 print(f"F1: {scores.mean():.3f} ± {scores.std():.3f}")
```

- Pipeline garante que o StandardScaler não veja o fold de teste
- Sem Pipeline, o scaler vazaria informação do teste para o treino

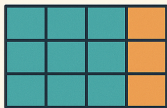
Listing 5: GridSearchCV e RandomizedSearchCV

```
1 from sklearn.model_selection import RandomizedSearchCV
2 from scipy.stats import randint
3 param_dist = {
4     "clf__n_estimators": randint(50, 300),
5     "clf__max_depth":    [None, 5, 10, 20],
6     "clf__min_samples_split": randint(2, 20)}
7 search = RandomizedSearchCV(
8     pipe,
9     param_distributions=param_dist,
10    n_iter=30,                # quantas combinações testar
11    cv=5,
12    scoring="f1_weighted",
13    n_jobs=-1,               # usa todos os cores
14    random_state=42)
15 search.fit(X_train, y_train)
16 print(search.best_params_)
17 print(f"Melhor F1 (CV): {search.best_score_:.3f}")
```

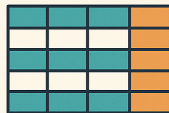


Cross-Validation

A Guide to Different Methods for Model Evaluation



K-Fold



Stratified K-Fold



Leave-One-Out



Leave-P-Out

Como funciona a Web



O que acontece ao acessar um site

1. Navegador envia **requisição HTTP** ao servidor
2. Servidor responde com **HTML** (estrutura)
3. Navegador baixa **CSS** (estilo) e **JavaScript** (comportamento)
4. Navegador *renderiza* a página — monta a árvore **DOM**

O que é o DOM?

- *Document Object Model* — representação em árvore do HTML
- Cada elemento HTML é um **nó** da árvore
- JavaScript e ferramentas de scraping

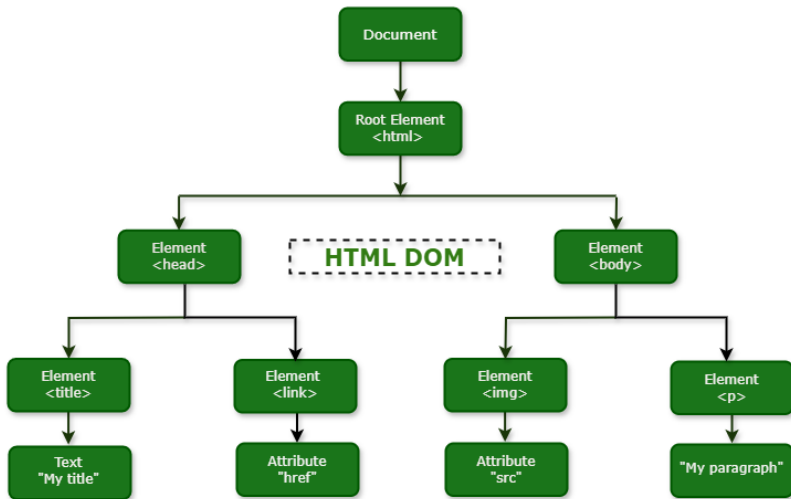
HTML mínimo

- `<tag>` abre, `</tag>` fecha
- `id` — identificador único por página
- `class` — grupo de elementos com estilo comum
- Atributos: `href`, `src`, `data-*`

CSS Selectors — a chave do scraping

- `div.produto` — `<div class='produto'>`
- `#preco` — elemento com `id='preco'`
- `table > tr > td` — `<td>` filho direto

Anatomia de uma página HTML



Quando o Web Scraping se torna necessário



Situações típicas

- Fonte **não oferece API** pública
- API existe, mas tem **limites de requisição** ou **paywall**
- Dados históricos não disponíveis para download
- Monitoramento de preços, notícias ou vagas em tempo real
- Pesquisa acadêmica em dados públicos da web

Antes de fazer scraping, verifique

- **robots.txt** — o site permite crawlers?
- **Termos de serviço** — há proibição explícita?
- Existe uma **API ou dataset** oficial?
- Os dados são **públicos** ou requerem login?

Atenção legal

Scraping de dados pessoais pode violar a LGPD. Scraping que sobrecarrega servidores pode configurar negação de serviço. Use com responsabilidade e respeite os limites.



Listing 6: Instalação e uso básico

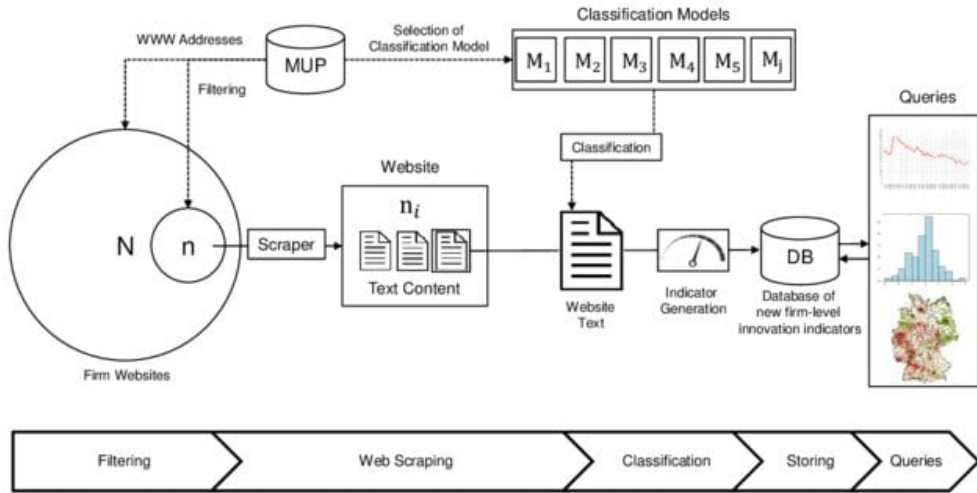
```
1 # pip install requests beautifulsoup4 lxml
2 import requests
3 from bs4 import BeautifulSoup
4
5 url = "https://pt.wikipedia.org/wiki/Inteligência_artificial"
6 resp = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
7 soup = BeautifulSoup(resp.text, "lxml")
8 # Título da página
9 print(soup.find("h1").text)
10
11 # Todos os parágrafos do artigo
12 paragrafos = soup.select("div.mw-parser-output > p")
13 for p in paragrafos[:3]:
14     print(p.get_text(strip=True))
15 # Todos os links internos da Wikipedia
16 links = [a["href"] for a in soup.find_all("a", href=True)
17          if a["href"].startswith("/wiki/")]
```



Listing 7: Tabelas HTML → DataFrame Pandas

```
1 import pandas as pd
2
3 url = "https://pt.wikipedia.org/wiki/Lista_de_pa%C3%ADses_por_PIB"
4 resp = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
5 soup = BeautifulSoup(resp.text, "lxml")
6
7 # pd.read_html usa BeautifulSoup internamente
8 tabelas = pd.read_html(resp.text)      # retorna lista de DataFrames
9 df_pib = tabelas[0]                   # primeira tabela da página
10 print(df_pib.head())
11
12 # Alternativa manual: navegar pela árvore
13 tabela = soup.find("table", class_="wikitable")
14 linhas = tabela.find_all("tr")
15 dados = [[td.get_text(strip=True) for td in tr.find_all("td")]
16           for tr in linhas[1:]]      # pula cabeçalho
17 df = pd.DataFrame(dados, columns=["pais", "pib", "ano"])
```

Exemplo completo: Wikipedia



Selenium — navegadores automatizados



Quando o conteúdo é gerado por **JavaScript** ou requer interação (login, clique, scroll), BeautifulSoup não basta — precisamos de um navegador real.

Listing 8: Configuração e navegação básica

```
1 # pip install selenium webdriver-manager
2 from selenium import webdriver
3 from selenium.webdriver.common.by import By
4 from selenium.webdriver.chrome.options import Options
5 opts = Options()
6 opts.add_argument("--headless")           # sem abrir janela
7 opts.add_argument("--no-sandbox")
8 driver = webdriver.Chrome(options=opts)
9 driver.get("https://quotes.toscrape.com/")
10 # Localizar elementos - mesmos seletores CSS
11 frases = driver.find_elements(By.CSS_SELECTOR, "span.text")
12 autores = driver.find_elements(By.CSS_SELECTOR, "small.author")
13 for frase, autor in zip(frases, autores):
14     print(f"{autor.text}: {frase.text}")
15 driver.quit()
```



Listing 9: Cliques, formulários e espera inteligente

```
1 from selenium.webdriver.support.ui import WebDriverWait
2 from selenium.webdriver.support import expected_conditions as EC
3 import time
4 driver.get("https://quotes.toscrape.com/login")
5 # Preencher formulário
6 driver.find_element(By.ID, "username").send_keys("user")
7 driver.find_element(By.ID, "password").send_keys("password")
8 driver.find_element(By.CSS_SELECTOR, "input[type='submit']").click()
9 # Esperar elemento aparecer (melhor que time.sleep fixo)
10 wait = WebDriverWait(driver, timeout=10)
11 wait.until(EC.presence_of_element_located(
12     (By.CSS_SELECTOR, "div.quote")
13 ))
14 # Scroll até o final da página (lazy loading)
15 driver.execute_script("window.scrollTo(0, document.body.scrollHeight);")
16 time.sleep(1) # aguardar carregamento após scroll
```

Técnicas anti-scraping — nem tudo são flores



Detecção de bots

- **Rate limiting** — bloqueia muitas requisições rápidas
- **CAPTCHAs** — reCAPTCHA v2/v3, hCaptcha
- **Fingerprinting** — tamanho de tela, plugins, fonte
- Detecção de webdriver no JavaScript
- Análise de padrão de comportamento (mouse, scroll)

Bloqueios técnicos

- **Cloudflare / WAF** — challenge automático
- **Conteúdo dinâmico** — dados só no JS, não no HTML
- **Tokens e cookies** — sessão expirada propositalmente
- **Honeypots** — links invisíveis que detectam bots
- Geração de seletores CSS aleatórios a cada deploy

Alternativas quando o scraping falha

Procure APIs não documentadas (aba Network do DevTools), datasets oficiais, arquivos de exportação ou parceiros de dados.



Listing 10: requests — a biblioteca padrão para HTTP

```
1 import requests
2 # GET simples
3 resp = requests.get("https://api.github.com/repos/pandas-dev/pandas")
4 resp.raise_for_status()           # lança exceção se status >= 400
5 dados = resp.json()              # deserializa JSON automaticamente
6 print(dados["stargazers_count"], dados["forks_count"])
7 resp = requests.get(
8     "https://api.github.com/search/repositories",
9     params={"q": "language:python stars:>10000", "sort": "stars"},
10    headers={"Authorization": "Bearer SEU_TOKEN"},
11 )
12 repos = resp.json()["items"]
13 page = 1
14 todos = []
15 while True:
16     r = requests.get(url, params={"page": page, "per_page": 100})
17     items = r.json()
18     if not items: break
19     todos.extend(items); page += 1
```



Listing 11: Boas práticas: sessão, retentativas e backoff

```
1 import requests, time
2 from requests.adapters import HTTPAdapter
3 from urllib3.util.retry import Retry
4 # Sessão com retentativa automática
5 session = requests.Session()
6 retry = Retry(total=3,
7               backoff_factor=1,          # espera 1s, 2s, 4s
8               status_forcelist=[429, 500, 502, 503])
9 session.mount("https://", HTTPAdapter(max_retries=retry))
10 # Respeitar Rate Limit via header
11 resp = session.get("https://api.exemplo.com/dados",
12                   headers={"X-API-Key": "TOKEN"})
13 limite = int(resp.headers.get("X-RateLimit-Remaining", 1))
14 reset = int(resp.headers.get("X-RateLimit-Reset", 0))
15 if limite == 0:
16     espera = reset - time.time()
17     print(f"Rate limit atingido. Aguardando {espera:.0f}s...")
18     time.sleep(max(0, espera))
```

Obrigado!

Alguma dúvida?

Agora vamos para os exercícios!